# Week 6 · Objects and lifecycles

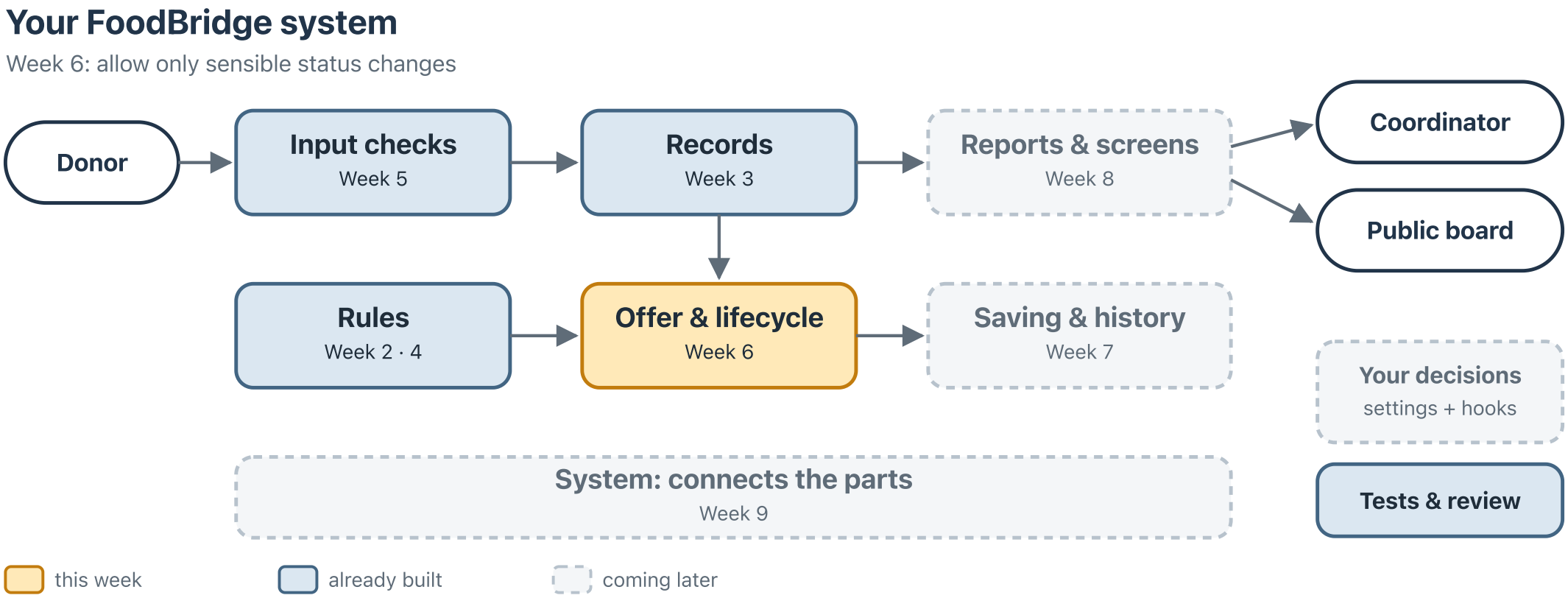
## Programming for Information Systems

Make wrong states hard to reach

# This week

| | |
|---|---|
| **Question** | How does a system keep each thing in a sensible state? |
| **Python** | reading a class, objects, attributes, methods, `self`; a dictionary of allowed moves |
| **Principle 6** | Make wrong states hard to reach |
| **You build** | `can_move(old_status, new_status)` |

# Your system so far



# The idea: entities, status, workflow

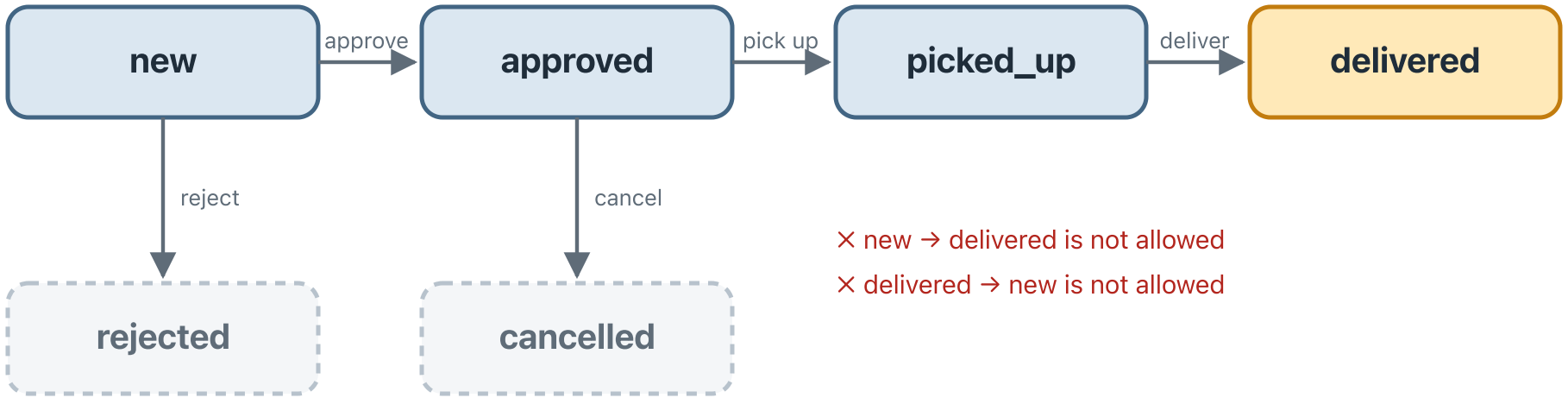
- An **entity** is one real thing the system follows: a parcel, a student, an offer.
- Its **status** says where it is on its journey.
- The **workflow** is the set of moves that make sense.

> A parcel cannot be *delivered* before it is *shipped*. A delivered parcel does not become *ordered* again.

# Every arrow is an allowed move

## The lifecycle of an offer

Only these moves are allowed. Everything else is refused.




✕ new → delivered is not allowed

✕ delivered → new is not allowed

# A class is a blueprint

```
class Parcel:
    def __init__(self, code, address):
        self.code = code
        self.address = address
        self.status = "ordered"

    def describe(self):
        return f"{self.code} to {self.address}: {self.status}"

parcel_a = Parcel("P01", "Battambang")
print(parcel_a.describe())        # P01 to Battambang: ordered
```

# `__init__`, `self`, attributes, methods

- `__init__` runs when you create an object. It sets the **starting** values.
- `self` means **this object**: `self.status` is this parcel's status.
- An **attribute** is a value on an object: `parcel_a.status`.
- A **method** is a function in the class: `parcel_a.describe()`.

```
parcel_a.status = "shipped"
print(parcel_a.describe())        # P01 to Battambang: shipped
print(parcel_b.describe())        # P02 to Siem Reap: ordered
```

# Read a real class: `bridge.Offer`

```
class Offer:
    def __init__(self, offer_id, donor, food, category, portions, hours_left, contact=""):
        self.id = offer_id
        ...
        self.status = "new"
        self.partner = ""

    def move_to(self, new_status, is_allowed):
        if not is_allowed(self.status, new_status):
            return False
        self.status = new_status
        return True
```

# Python does not stop you

```
offer = bridge.Offer("F101", "Sunrise Bakery", "Bread rolls", "bread", 30, 5)
offer.status = "deliverd"          # a typo, and every step skipped
print(offer.is_open())             # False
```

- Not open: it disappears from the coordinator's list.
- Not "delivered": it is not counted as delivered either.

> Python cannot make wrong states **impossible**. Good design makes them **hard to reach**.

# A dictionary of allowed moves

```
NEXT_LIGHT = {"green": ["yellow"], "yellow": ["red"], "red": ["green"]}
print("yellow" in NEXT_LIGHT["green"])     # True
print("red" in NEXT_LIGHT["green"])        # False

print(bridge.ALLOWED_MOVES["approved"])    # ['picked_up', 'cancelled']
print(bridge.ALLOWED_MOVES["delivered"])   # []  an end state
```

- Each key is a status; each value is a **list** of next statuses.
- The workflow is **data**: change the dictionary, not the logic.

# `move_to` asks the rule first

```
curry = bridge.Offer("F102", "Lotus Cafe", "Vegetable curry", "rice meals", 12, 4)
print(curry.move_to("delivered", bridge.models.can_move), curry.status)  # False new
print(curry.move_to("approved", bridge.models.can_move), curry.status)  # True approved
```

- The rule is a function, passed as a value (like hooks and `bridge.check`).
- A refused move **changes nothing** and returns `False`.
- The rule sits **next to the data** it protects.

# Practice: your own class

```python
class PickupTask:
    def __init__(self, offer_id, volunteer, portions):
        self.offer_id = offer_id
        self.volunteer = volunteer
        self.portions = portions

    def describe(self):
        return f"{self.offer_id}: {self.volunteer} collects {self.portions} portions"
```

- ⭐⭐ Stretch: add `status` and `move_to` with `waiting → on_the_way → done`.

# ⚖️ Good or risky?

**Version A** (from an AI assistant)

```
def update_status(offer, typed):
    offer.status = typed.strip().lower()

def mark_delivered(offer):
    offer.status = "delivered"

def fix_mistake(offer):
    offer.status = "new"          # reset
```

"Buttons for volunteers, and a way to fix mistakes."

**Version B**

```
def update_status(offer, new_status, is_allowed):
    old = offer.status
    if offer.move_to(new_status, is_allowed):
        return "done: " + old + " -> " + new_status
    return "refused: " + old + " -> " + new_status
```

One door, one rule, a clear answer.

# What Version A does to one offer

| Step | Status now | Who is harmed |
|---|---|---|
| one click on "delivered" | delivered | nobody collects the curry; it spoils, and the delivered total is false |
| "fix a mistake" | new | a volunteer drives for food that is gone; portions counted twice |
| a volunteer types `Deliverd` | deliverd | the offer vanishes from every list and total |

- No answer is returned, and there is no single rule to read or test.

> Version B is better, but Python still allows `offer.status = ...` elsewhere. One door, and review the code.

# Principle 6 · Make wrong states hard to reach

1. Allow **only** the moves that make sense.
2. Keep the rule **next to the data**: `move_to` asks before it changes.
3. Change the status **only** through that one door.

> Python cannot make wrong states impossible, so the team agrees on one door and reviews code for others.

# You build: `can_move(old_status, new_status)`

```
ALLOWED_MOVES = {
    "new": ["approved", "rejected"],
    "approved": ["picked_up", "cancelled"],
    # TODO 1: picked_up, delivered, rejected, cancelled
}
# TODO 2: if not REQUIRE_APPROVAL, new may go straight to picked_up

def can_move(old_status, new_status):
    # TODO 3: unknown old_status -> False
    # TODO 4: True only when new_status is in its list
    return False
```

# One decision, two places

```
REQUIRE_APPROVAL = True
MY_SETTINGS = {"REQUIRE_APPROVAL": REQUIRE_APPROVAL}

if not REQUIRE_APPROVAL:                          # build the dict FROM the setting
    ALLOWED_MOVES["new"].append("picked_up")

bridge.check("can_move", can_move, MY_SETTINGS)
```

- The check asks `can_move("new", "picked_up")`: `False` with approval, `True` without.
- If the setting and the dictionary disagree, the system says one thing and does another.

# Plug it in

```
my_system = bridge.System(MY_SETTINGS, hooks={"can_move": can_move})
my_system.run_day()
ok, message = my_system.move("F004", "delivered")    # deliver before pickup
print(ok, message)     # False F004 cannot move from approved to delivered.
```

- In the notebook: 🚫 ✅ ✅ 🚫 🚫 for five tries.
- Each 🚫 is a wrong state your rule kept out; the record did not change.

# 🧭 Your decision · approve before pickup?

- `REQUIRE_APPROVAL = True`: a coordinator checks the partner can take the food.
- `REQUIRE_APPROVAL = False`: urgent food can be collected at once.
  - The system prints a ⚠️ **risky setting** warning.
  - Your decision log **must** justify it.

> Who is protected by the wait, and who loses food because of it?

# This week's tasks

- ⭐ **Core:** P1–P4, the ⚖️ discussion, `can_move` with all ✅, plugged in, decision log.
- ⭐⭐ **Stretch:** a lifecycle for `PickupTask`; `next_moves(status)` so screens show only allowed buttons.
- 🔎 **Research 5 (5%):** keeping data safe: backups, audit logs, transactions. Two updates that must happen together; what goes wrong between them; how to check before and after. Share it in the Week 7 warm-up.

# Next week · Saving and history

Right now everything disappears when the notebook closes. You will **save** records safely and keep a **history** of every move.